T E A M O L I V E

Prototype Game Design Document

GAM 541 · Prototype Milestone

Rahul Nair · Dong Ta · Jackey Cen · Sumin · Josh

1. Purpose & Overview

This document describes the three prototypes produced by Team Olive during the Prototype Milestone of GAM 541. Each prototype was designed as a focused feasibility test for a specific core mechanic intended for the team's final rhythm-action game. Rather than building toward a polished product, each prototype exists to answer a clearly defined design question and to surface technical and experiential risks before full production begins.

The three prototypes are:

- Prototype 1 — Explicit Rhythm Game: Can explicit UI-driven beat feedback create a satisfying rhythm-action loop?
- Prototype 2 — Implicit Rhythm Game: Can rhythm be communicated through the environment and rewarded subtly rather than enforced strictly?
- Prototype 3 — Enemy Behavior: Can rhythm-based enemy attacks be implemented with a flexible, modular AI system?

2. Shared Systems

All three prototypes were built on top of the custom game engine developed during the Engine Proof milestone. Two engine systems are central to the rhythm mechanics across prototypes:

2.1 Conductor System

The Conductor System is the authoritative source of beat timing in the engine. It tracks the current BPM, advances an internal beat clock synchronized with audio playback, and fires beat events that other systems subscribe to. All rhythm-dependent gameplay — from input gating to camera effects — is driven by events dispatched by the Conductor rather than by ad-hoc timers. This ensures that every prototype shares a single, consistent definition of when a beat occurs, regardless of the song or scene.

2.2 Beat Bar Component

The Beat Bar Component is a UI element that visualizes the current beat position as a moving line travelling across a horizontal bar. The center of the bar represents the target beat window — the moment at which the player's input is considered on-beat. The component subscribes to

© 2026 DigiPen (USA) Corporation

the Conductor System and updates its visual state in lockstep with beat events. It was used directly in Prototype 1 as the primary input cue, and its underlying timing data informed the implicit feedback systems explored in Prototype 2.

PROTOTYPE 1 Explicit Rhythm Game

Feasibility question: Can explicit UI beat feedback drive a satisfying rhythm-action loop?

Overview

Prototype 1 is a traditional, action-oriented rhythm platformer with explicit visual cues for beat synchronization. The design deliberately surfaces the rhythm system to the player in the most direct possible way: a visible beat bar drives all high-stakes actions, and the player must read and respond to it in order to perform jumps and dashes. This prototype was built to validate that the Conductor System and Beat Bar Component, in their most transparent form, produce a readable and responsive gameplay experience.

Design Question

Does enforcing jump and dash exclusively on the beat — communicated through a UI beat line — feel fair, readable, and engaging?

Core Mechanics

Beat Bar & Input Gating

A horizontal beat bar is displayed prominently on screen. A moving indicator travels across the bar from left to right, driven by the Conductor System at the current song BPM. When the indicator reaches the center zone of the bar, the player is within the valid beat window. Jump (Space) and Dash (Shift) inputs are only registered if the player presses them while the indicator is inside this window. Inputs outside the window are silently rejected — the character does not react, and no negative feedback is given beyond the missed opportunity.

Movement

Horizontal movement via the Arrow Keys is unrestricted and does not require beat synchronization. This provides a baseline of player agency at all times and ensures the prototype does not feel uncontrollable, while still making the beat the gating condition for the more expressive actions.

Controls

Input	Action	Beat Required?
Arrow Keys (Left / Right)	Move character horizontally	No — free movement
Space	Jump	Yes — beat window required
Shift	Dash	Yes — beat window required

© 2026 DigiPen (USA) Corporation

Visual Feedback

- Beat bar indicator moves continuously across the screen, providing constant visual rhythm reference.
- The center zone of the beat bar acts as the target; no additional flash or sound cue is used in this prototype (POC level).
- Failed inputs produce no visual response, keeping the feedback loop clean for this feasibility test.

Prototype Goals & Findings

The primary goal was to confirm that input gating tied to a visible beat bar produces a readable and responsive feel. The prototype confirmed that the Conductor System fires beat events with sufficient precision to make the input window feel fair. The Beat Bar Component provided an effective visual anchor. Key open questions carried forward into Prototype 2: Does explicit gating feel too punishing? Would removing the hard gate while still rewarding on-beat actions produce a more enjoyable experience?

PROTOTYPE 2 Implicit Rhythm Game

Feasibility question: Can rhythm be felt through the environment and reward — without enforcing it?

Overview

Prototype 2 explores a softer approach to rhythm integration. The explicit beat bar is removed entirely. Instead, the player is invited to feel the rhythm through environmental cues, and on-beat actions are rewarded with enhanced performance rather than gated. Jump and dash are available at any time, but pressing them in sync with the beat yields a noticeably stronger result. This prototype tests whether implicit feedback — reward rather than restriction — can motivate players to naturally seek the beat.

Design Question

If the player is never blocked from acting, but on-beat actions feel distinctly more powerful and satisfying, will players organically gravitate toward rhythmic play?

Core Mechanics

On-Beat Boost

Jump and Dash are available without restriction. When either action is performed within the beat window (as defined by the Conductor System), it receives a boost: jumps travel higher and dashes cover more distance. This creates a skill ceiling — players who internalize the rhythm gain a meaningful advantage — without creating a hard barrier for players still learning the timing.

Implicit Beat Cue (POC)

Rather than a dedicated beat bar, the prototype uses a UI element already present in the game — the lives hearts in the top corner of the screen — as a proof-of-concept beat pulse indicator. The hearts subtly animate or flash in sync with the Conductor System, providing a non-intrusive

© 2026 DigiPen (USA) Corporation

environmental signal of the current beat. This approach was chosen to test whether a diegetically motivated cue (something that belongs in the game world's UI) can serve as an adequate rhythm reference without breaking immersion.

Camera Feedback (POC)

A camera effect fires in sync with on-beat successful actions, providing a positive sensory signal that the player has hit the beat correctly. This acts as a proof-of-concept for a positive signifier — a piece of feedback that communicates success rather than failure, reinforcing the rhythm loop through reward rather than punishment.

Controls

Input	Action	Beat Required?
Arrow Keys (Left / Right)	Move character horizontally	No — free movement
Space	Jump (standard) / Jump+ (on beat)	No gate — boosted on beat
Shift	Dash (standard) / Dash+ (on beat)	No gate — boosted on beat

Visual Feedback

- Lives hearts in the top corner pulse in sync with the beat (POC implicit cue).
- Camera effect triggers on successful on-beat actions as a positive signifier (POC).
- No negative feedback for off-beat inputs — the player simply receives the non-boosted version of the action.

Prototype Goals & Findings

The core question was whether removing the hard gate would make the game feel more accessible while still motivating rhythmic play. The prototype confirmed that the on-beat boost is perceivable and that the camera effect successfully communicates when the player has hit the beat. The lives hearts as implicit cue were a functional POC but highlighted the need for a more deliberate environmental cue design in the final game. The removal of the hard gate made the prototype feel noticeably more approachable than Prototype 1.

PROTOTYPE 3 Enemy Behavior

Feasibility question: Can rhythm-based enemy attacks be implemented with a modular, adaptive AI system?

Overview

Prototype 3 shifts focus to the enemy side of the rhythm-action equation. Rather than testing how the player interacts with the beat, this prototype investigates whether enemy behavior can be made rhythm-aware, modular, and adaptive using a Goal-Oriented Action Planning (GOAP) framework. The prototype takes the form of a boss encounter set in a deliberately minimal level,

© 2026 DigiPen (USA) Corporation

keeping the environment uncluttered so that the enemy behavior itself is the primary subject of evaluation.

Design Question

Can a GOAP-based enemy AI produce behavior that is rhythm-aware, context-sensitive, and modular enough to be reused across different encounter types without significant rework?

Level Design Rationale

The boss fight level was intentionally kept blank — a flat, open space with no environmental obstacles. This decision was made deliberately to isolate the enemy behavior as the sole source of challenge. A cluttered level would obscure whether difficulty was coming from the AI or from the environment, making it harder to evaluate the behavior system in isolation.

Enemy Behavior System — GOAP Framework

Architecture

The Enemy Behavior System implements a modular Goal-Oriented Action Planning (GOAP) framework. Rather than relying on rigid behavior trees or scripted sequences, the system dynamically evaluates the enemy's current world state and selects a short action plan that best satisfies its current goals.

World State Evaluation

The planner evaluates a set of world conditions at runtime to inform decision-making:

- Enemy health
- Distance to the player
- Per-action cooldown states
- Current phase of the encounter

Action Structure

Each action in the action list defines:

- Preconditions — the world state requirements that must be met for the action to be considered.
- Effects — how the world state changes if the action is executed.
- Cost — a numeric weight used by the planner to compare competing action sequences.

This structure keeps individual actions simple and self-contained. Actions include behaviors such as moving toward the player, performing a melee attack, executing a ranged attack, dodging, and retreating. Because each action is independent, new behaviors can be added to the roster without modifying existing ones.

Planning & Execution

At each planning interval, the system simulates possible future world states by chaining actions up to a defined search depth. The sequence with the lowest cumulative cost that satisfies the enemy's current goal is selected and executed. This allows the boss to exhibit adaptive, context-sensitive behavior — retreating when health is low, pressing the attack when the player is within range, and varying its approach based on cooldown availability.

© 2026 DigiPen (USA) Corporation

Rhythm Integration

Enemy attacks are scheduled to fire on beat, synchronized with the Conductor System. This means the GOAP planner does not simply pick the best action at any moment — it also defers attack execution to the next available beat window, making enemy behavior feel musically grounded. The rhythm-aware attack timing was a key proof-of-concept for the final game, where players will need to anticipate and respond to enemy attacks in time with the music.

Modularity & Reuse

A core design goal was to ensure the enemy code was modular enough to be brought into other prototypes and the final game as a plug-and-play component. Each action item is an independent class with a defined interface, and the planner itself is decoupled from any specific enemy type. This means the same framework can drive simple enemies with a small action set or complex bosses with a large repertoire, simply by varying the action list provided to the planner.

Prototype Goals & Findings

The prototype confirmed that the GOAP framework can produce believable, adaptive boss behavior within the constraints of a rhythm game. Rhythm-synchronized attack timing worked as intended, with the boss's attacks feeling musically motivated. The blank level design was validated as the right choice for this phase — it allowed the team to focus entirely on behavior tuning without environmental noise. The modular action list structure proved its value during development, as adding and adjusting individual behaviors required no changes to the planner itself. Key open questions for production: how should enemy rhythm attacks telegraph to the player, and how many simultaneous enemies can the planner support before performance becomes a concern?

4. Summary & Next Steps

All three prototypes achieved their primary feasibility goals. The Conductor System and Beat Bar Component proved to be reliable foundations for rhythm-driven gameplay. The comparison between Prototype 1 and Prototype 2 provided actionable data on the trade-offs between explicit gating and implicit reward, and Prototype 3 confirmed that a modular GOAP-based AI can be made rhythm-aware without sacrificing flexibility.

The following questions are carried forward into the next phase of development:

- What is the right balance between explicit beat cues and implicit environmental feedback in the final game?
- How should the implicit beat cue be designed to be more intuitive than the lives-hearts POC?
- How should enemy rhythm attacks telegraph their timing to the player in a way that is readable but not trivially avoidable?
- What is the performance ceiling of the GOAP planner with multiple simultaneous enemies?
- How do the on-beat boost values (jump height, dash distance) need to be tuned to feel rewarding without being overpowered?

© 2026 DigiPen (USA) Corporation